DP Concepts & Questions

Video 40

Think of your skills as a system you are constantly upgrading; every new thing you learn helps you handle bigger challenges. Do not let this year pass without adding valuable tools to your toolkit. Put in the daily effort to upskill, and you will be amazed at what you can build.

MIK

codestorywithmik

CSwithMIK

codestorywithMIK

- 1-D based DP
- Grid based DP
- String based DP
- Knapsack Series
- Digit DP
- Game Strategy

We'll do :-

(·) RECURSION + MEMOIZATION (Top Down)

(·) Bottom Up.

(·) Time & Space

Knapsack Series

(video - 5)

Variants

- Fractional Knapsack
- 0/1 Knapsack
- Unbounded Knapsack

0-1 Knapsack

Bottom-up (optimised)

First let's understand when to

First, let's think and how to optimize ...

video-4

```cpp
class Solution {
 public:
  int knapsack(int W, vector<int> &val, vector<int> &wt) {
      int n = val.size();

      vector<vector<int>> t(n+1, vector<int>(W+1, 0));
      //t[i][j] = max profit with i items and j knapsack weight
      //return t[n][W];

      //Base case : if(n == 0 ) return 0; //no item
      for(int j = 0; j < W+1; j++) {
          t[0][j] = 0;
      }

      //Base case : if(W == 0) return 0; //knapsack full
      for(int i = 0; i < n+1; i++) {
          t[i][0] = 0;
      }

      for(int i = 1; i < n+1; i++) {
          for(int j = 1; j < W+1; j++) {

              int take = 0;
              int skip = 0;

              //take
              if(wt[i-1] <= j) {
                  take = val[i-1] + t[i-1][j - wt[i-1]];//val[i-1] + solve(i-1, j - wt[i-1], val, wt);
              }

              skip    = t[i-1][j]; //solve(i-1, j, val, wt);

              t[i][j] = max(take, skip); //return t[n][W] = max(take, skip);

          }
      }

      return t[n][W];
  }
};
```

Prev row

take = val[i-1] + t[i-1][j - wt[i-1]];

skip = t[i-1][j];

t[i][j] = max(take, skip);

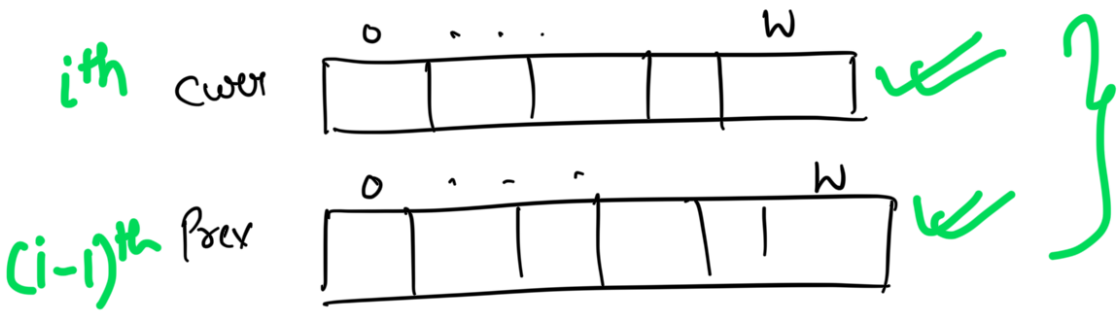
weight of items

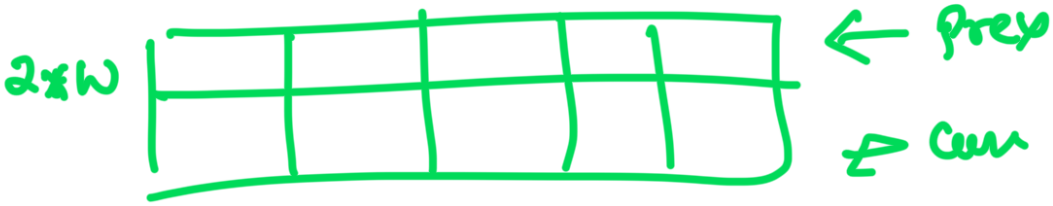
	0	1	.	.	.	.	W
	✗	✗	✗	✗	✗	✗	
	✗	✗	✗	✗	✗	✗	
Prev	✓	✓	✓	✓	–	–	
Curr							
.							
n							

no of items

n×W

i^{th} row is only dependent on $(i-1)^{th}$ row





Always use the existing code for space optimization :-

```
ass Solution {
ublic:
  int knapsack(int W, vector<int> &val, vector<int> &wt) {
      int n = val.size();
                                  n.W
      vector<vector<int>> t(n+1, vector<int>(W+1, 0));
      //t[i][j] = max profit with i items and j knapsack weight
      //return t[n][W];

      //Base case : if(n == 0 ) return 0; //no item
      for(int j = 0; j < W+1; j++) {
          t[0][j] = 0;
      }

      //Base case : if(W == 0) return 0; //knapsack full
      for(int i = 0; i < n+1; i++) {
          t[i][0] = 0;
      }


      for(int i = 1; i < n+1; i++) {
          for(int j = 1; j < W+1; j++) {

              int take = 0;
              int skip = 0;

              //take
              if(wt[i-1] <= j) {
```

vector<int> curr (W+1, 0); i

vector<int> Prev (W+1, 0); //i-1

take = val[i-1] + Prev[j - wt[i-1]];

skip = Prev[j];

curr[j] = max(take, skip);

```
            take = val[i-1] + t[i-1][j - wt[i-1]];//val[i-1] + solve(i-1, j - wt[i-1], val, wt);
        }

        skip    = t[i-1][j]; //solve(i-1, j, val, wt);

        t[i][j] = max(take, skip); //return t[n][W] = max(take, skip);

    }
  }

  return t[n][W];
}
```

2D

prev = curr;

return prev[W];

1D

T.C = O(n·W)

S.C = O(W)